

- **Business Engagement:** Effective collaboration depends upon meaningful and productive engagement between technical teams and the business people they support.
- **Team Structures:** Collaboration also depends on ensuring that the most appropriate team structures are in place to support effective teamwork, connecting the people with the right skills and empowering them to work as self-organizing, cross-functional feature teams.
- **Capability Foundations**
 - **Test Techniques:** Applying appropriate and effective test techniques, both in the design of formal test cases and in the dynamic exploration of the product, will contribute to Better Test Design and Greater Coverage.
 - **Skills and Experience:** The effectiveness of those test techniques (and other foundations) is heavily dependent upon the skills and experience within the team.
- **Technical Foundations**
 - **Effective Automation:** The ability to quickly and efficiently automate the majority of defined tests is critical to supporting the pillars of Faster Feedback and Greater Coverage. This in turn is dependent upon the testability of the architecture and strong configuration management.
 - **Configuration Management:** The discipline of managing versions of tests and other code, the controlled configuration of test environments and data, and the identification of all relevant artifacts in the software development and test process.
 - **Testability:** The extent to which the architecture of the system under test is designed to support automated testing. The characteristics of a testable system align with good architectural goals, such as loose coupling, dependency injection, strict separation of concerns, a well-defined domain model, and ubiquitous use of domain language.

The Foundation areas are all built upon the Organizational Bedrock, representing the leadership style and culture of the organization:

- **Management and Leadership:** The ability to hire and inspire great people, to remove or overcome organizational constraints and impediments that hinder them from doing their best work, to see and exploit the opportunities for people to excel, and to motivate individuals to achieve what is best for the team.

- **Engineering Discipline:** The guidance and support given to ensure that technical teams have the right tools and technical environment at their disposal, are empowered to make the most of them, and are given the education and personal development to ensure that their skills are always current and appropriate.
- **Culture and Values:** Ultimately, the whole environment and structure for success are built upon the culture and values of the organization.

Testing forms a big part of any team's ability to deliver business value frequently at a sustainable pace. We know we can't test quality in; we have to build it in from the start. When we begin software development with testing, we're more likely to build the right value for our customer, and when we complete the development and delivery with testing, we help ensure that we have built that value right.

SUMMARY

We encourage people to remember the seven key success factors for agile testing: use the whole-team approach, maintain an agile testing mindset, automate regression tests, focus on feedback, use core practices, collaborate, and keep the big picture in mind. In the spirit of continual learning, we encourage teams to experiment with the confidence-building practices we discussed in this book:

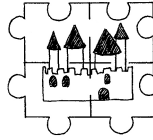
- Driving development with examples



- Exploratory testing



- Feature testing



- Continual learning



- Context sensitivity



- Keeping it real



Strive to achieve confidence by applying the Pillars of Testing. It's the best thing we can do for our customers.

PAGE OBJECTS IN PRACTICE: EXAMPLES

In Chapter 16, “Test Automation Design Patterns and Approaches,” Tony Sweets explained the concepts behind using the Page Object pattern for designing automated tests. Here are code snippets and technical details so that you can try this out yourself.

AN EXAMPLE WITH SELENIUM 2—WEBDRIVER

The following piece of code is a Page Object for the Wikipedia home page. It is written in Java using Selenium 2 (WebDriver). This particular Page Object has two public methods (or actions) it can perform. We can either click the privacy link, or we can search Wikipedia using the term “Nissan Motor Company.” These two methods (plus the initializer method, which is known as the constructor) make up this Page Object’s public interface. This is the interface you want to keep as stable as possible, and it is what your test script will use to navigate the site and to perform tests.

If you think you lack the programming skills required to implement this object, the programmers on the team could write the code for the testers and leave the test scripts for the team members who are actually specifying the tests.

One thing you should notice about this code is that we encapsulated the elements of the page and made them private. This helps to avoid brittle tests, as you don’t have direct access to them, and they are what is most likely to change as time goes by. The only way to access them is

through the public interface, which is intentionally vague. When your test script calls `searchForNissan()`, it has no idea how the Page Object will perform that piece of work, nor should it. Since we are hiding the implementation details away from the actual test, we can thus change the implementation without changing the test.

For example, let's say Wikipedia made a system update and searching for "Nissan Motor Company" no longer returns the real Nissan page. Instead it returns a blocker page, with an image of founder Jimmy Wales on it asking for a donation, and that page has a link to the real Nissan page. Without changing your test, you could modify the `NissanPage` Page Object to click the link below the donation request before returning the `NissanPage` object back to the test.

```
// This is the Home Page Class which when created (when
// the test is run) becomes a Page Object
public class HomePage {
    // This is a logging utility. Used to log to a file
    // and/or the screen
    private static final Logger log = LoggerFactory.
getLogger(HomePage.class);

    // This is the Object that acts as a web browser when
    // using Html Unit
    // or the thing that "drives" a real web browser like
    // Firefox
    private WebDriver webDriver;

    // These are elements (Links, Input Boxes, etc.) on
    // this page that I need to use
    private WebElement privacyPolicyLink;
    private WebElement searchBox;
    private WebElement submitButton;

    // Constructor of this Page Object
    public HomePage(WebDriver webDriver) {
        log.info("Page = {}", webDriver.getTitle());
        log.info("URL = {}", webDriver.getCurrentUrl());
        this.webDriver = webDriver;

        // Test that the page we are on is actually
        // the Wikipedia home page.
        // We know that we are on the home page if
        // the title matches the single
        // word "Wikipedia" and nothing else
        if (!webDriver.getTitle().equals("Wikipedia")) {
            throw new IllegalStateException("Requested
HomePage: current page - " + webDriver.getTitle());
        }
    }
}
```

```

        //Initialize WebElements
        privacyPolicyLink =
webDriver.findElement(By.partialLinkText("Privacy Policy"));
        searchBox = webDriver.findElement(By.
id("searchInput"));
        submitButton = webDriver.findElement(By.
name("go"));
    }

    public PrivacyPolicyPage clickPrivacyPolicyLink() {
        privacyPolicyLink.click();

        return new PrivacyPolicyPage(webDriver);
    }

    public NissanPage searchForNissan() {
        searchBox.sendKeys("Nissan Motor Company");
        submitButton.submit();

        return new NissanPage(webDriver);
    }
}

```

Let's look at a test script (I use that term loosely, as this is a real Java program/test). This is Java code and it's implemented as a JUnit. Using the JUnit framework allows us to take advantage of all of the test-friendly add-ons and ecosystem for free. For example, the JUnit output will be displayable by continuous build systems, by default, with no extra work needed. You can also run this with a simple Maven command just like any other test.

I consider this a script, because it is a top-down design. It's not really object oriented, but it uses Page Objects that you have created to represent your system under test. You need to create a WebDriver object that is part of Selenium 2, initialize it with the web browser it is going to control (HtmlUnit in this case), and point it to a URL. Then you simply hand it off to your Page Object that represents that specific URL, HomePage in this case. I'm also logging along the way because if your test breaks, you will want to know what step it was on so that you can manually investigate what happened.

```

@Test
public void testWikipedia() {
    WebDriver webDriver = new HtmlUnitDriver();

    log.info("Requesting HomePage");
    webDriver.get("http://www.wikipedia.org");
    HomePage homePage = new HomePage(webDriver);
    Assert.assertNotNull(homePage);
}

```

```

        log.info("Requesting Privacy Policy");
        PrivacyPolicyPage privacyPolicyPage = homePage.
clickPrivacyPolicyLink();
        Assert.assertNotNull(privacyPolicyPage);

        // If I go back I should be at the home page again
        log.info("Hitting the Back button");
        webDriver.navigate().back();
        HomePage homePageAfterBack = new
HomePage(webDriver);
        Assert.assertNotNull(homePageAfterBack);

        // Search for Nissan
        log.info("Looking up Nissan Motor Company");
        NissanPage nissanPage = homePageAfterBack.
searchForNissan();
        Assert.assertNotNull(nissanPage);

        // Go to the list of Nissan cars
        log.info("Clicking List of Nissan Cars");
        NissanCarListPage nissanCarListPage = nissanPage.
selectListOfCars();
        Assert.assertNotNull(nissanCarListPage);

        // Finally go to the Nissan Skyline GT-R Page
        log.info("Clicking Skyline GT-R");
        NissanSkylinePage nissanSkylinePage =
nissanCarListPage.selectNissanSkyline();
        Assert.assertNotNull(nissanSkylinePage);

        webDriver.quit();
    }

```

JUnit uses the Assert class to check aspects of your code. You can check for nulls, equality, and true/false, among others. If the assertion fails, the script stops at that point, and it is logged as test failure automatically. This can show up in your build system, on the command line, or in a report. At the very end of the test script we send a quit command to the WebDriver object, which cleans up any resources being used. This is specifically useful when using a real browser like Internet Explorer or Firefox.

USING THE PageFactory CLASS

What we have seen already is just plain Page Objects implemented with Selenium 2. We could go a step further with Selenium 2 and use the built-in PageFactory helper class that it provides. This is an implementation of a widely used design pattern known as the Factory pattern.

The PageFactory helps you out by giving simple-to-read annotations for locating web page elements. Annotations are a relatively new feature to Java, and they always start with the ampersand (@). The PageFactory does the creation of the Page Object for you and initializes the elements based on the annotations you wrote to locate the item. Here is the HomePage object again with the PageFactory implementation:

```
public class HomePage {
    private static final Logger log = LoggerFactory.
getLogger(HomePage.class);
    private WebDriver webDriver;

    @FindBy(how = How.PARTIAL_LINK_TEXT, using = "Privacy
Policy")
    private WebElement privacyPolicyLink;
    @FindBy(how = How.ID, using = "searchInput")
    private WebElement searchBox;
    @FindBy(how = How.NAME, using = "go")
    private WebElement submitButton;

    public HomePage(WebDriver webDriver) {
        log.info("Page = {}", webDriver.getTitle());
        log.info("URL = {}", webDriver.getCurrentUrl());
        this.webDriver = webDriver;
        if (!webDriver.getTitle().equals("Wikipedia")) {
            throw new IllegalStateException("Requested
HomePage: current page - " + webDriver.getTitle());
        }
    }

    public PrivacyPolicyPage clickPrivacyPolicyLink() {
        privacyPolicyLink.click();
        return PageFactory.initElements(webDriver,
PrivacyPolicyPage.class);
    }

    public NissanPage searchForNissan() {
        searchBox.sendKeys("Nissan Motor Company");
        submitButton.submit();
        return PageFactory.initElements(webDriver,
NissanPage.class);
    }
}
```

Notice that not much has changed, but it is enough that you probably do not want to mix the two styles, as your test scripts are slightly different as well. Your test script never actually creates a Page Object on its own anymore. It delegates to the factory. You are not buying much with

this change; however, it is slightly cleaner to look at, and the annotations make it easier for you to write and manage the Page Object.

The test script has to change as well. When we created the Page Object ourselves it looked like this:

```
log.info("Requesting HomePage");
webDriver.get("http://www.wikipedia.org");
HomePage homePage = new HomePage(webDriver);
Assert.assertNotNull(homePage);
```

Now with the PageFactory creating the object, it looks like this (changes are highlighted):

```
log.info("Requesting HomePage");
webDriver.get("http://www.wikipedia.org");
HomePage homePage = PageFactory.  
initElements(webDriver, HomePage.class);  
Assert.assertNotNull(homePage);
```

The full code for this example is located on my GitHub account (Sweets, 2013). The README file has all the information needed to run this code on your local box.

If you are on a Mac with the developer tools installed, you should be able to run these commands to download and run this suite of tests:

```
> cd /tmp
> git clone https://github.com/tsweets/page-objects.git
> cd page-objects/
> mvn clean test
```

This will work on any platform given you have Java, Git, and Maven installed. The temp directory is system specific, however, with /tmp being part of the Mac and other UNIX/Linux-based boxes. This will run three versions of the same test. It will run a brittle version, the Page Object version, and the page factory version.

PROVOCATION STARTERS

Carol Vaage used a list of questions to get her grade 1 students to start thinking about the world and how they interacted with it. These are questions that can be used to think about your product, and how you may use them to elicit examples or uncover assumptions. See the bibliography for Part II, “Learning for Better Testing,” for the link.

I wonder . . .

Could you show me . . .

Let’s figure out how that could be . . .

So, in other words, if you . . . then . . . ?

What would happen next?

But why would that be?

So, now our question is . . .

Did you notice?

What have you discovered . . . ?

I know you can figure this out—go work with your team and see what ideas might work.

This (person/group) has run into a problem. What can we think of to help them solve it?

Why do you think that happened?

Describe what you see.

What does that look like to you?

How does that happen?

Why did it work that way?

How can you tell?

How is this different?

What do you think will happen?

How can you find out?

Give it a try and let me know how it turns out.

Can you show me the evidence?

What is your theory?

GLOSSARY

Acceptance test Acceptance tests are tests that define the business value each story (or feature) must deliver and help define the scope of the story (or feature). They may verify functional requirements or non-functional requirements such as performance or reliability. Although they are used to help guide development, they are done at a higher level than the unit-level tests used for code design in test-driven development. This is a broad term that may include both business-facing and technology-facing tests.

Acceptance-test-driven development (ATDD) Teams doing ATDD specify executable acceptance tests using examples elicited from conversations with customers. Each example is discussed and, if possible, specified in an executable format. Programmers use the tests as guidance to develop the feature. Once code is written to make a test pass, it may be kept as an automated regression test, documenting feature behavior.

Application life cycle management (ALM) ALM refers to managing an application product's life cycle by people in multiple disciplines working through the diverse activities involved in product delivery. Agile ALM encompasses activities such as continuous integration and delivery, collaborating with customers to describe examples that guide development, coding, testing, and managing releases.

Behavior-driven development (BDD) Examples of desired behavior from conversations with stakeholders are turned into executable tests in a `given_when_then` format. Programmers automate the tests and

then write the code to make each test pass. BDD can be used at the unit level for test-driven design and can be used for business-facing tests that encompass multiple components and layers of the application and verify business-facing quality.

Build-measure-learn (BML) Build-measure-learn is a term from lean startup in which a team turns an idea into a deliverable product, measures customer response, and learns whether the idea should be pursued as is, changed in some way, or abandoned.

Build pipeline When code is checked into the source code control system, the commit kicks off a build pipeline. For more mature teams, this means running a quick suite of unit-level regression tests to provide fast feedback, followed by a number of test suites at the API and user interface levels to detect regression failures. Build pipelines may be configured to automatically deploy build artifacts to test, staging, and production environments.

Continuous integration (CI) In teams using the practice of continuous integration, team members check in code changes frequently throughout the day. Each check-in triggers an automated build process in which automated regression tests provide fast feedback to verify whether the code changes have introduced regression failures. These builds produce artifacts that may be automatically or manually deployed to different environments such as test, staging, and production.

Customer team Customer teams include all stakeholders outside of the software delivery team, such as product owners, product managers, business experts, subject matter experts, and end users.

Cycle time In the generic sense, it is the time from when something starts to when it finishes. Specifically, we mean the elapsed time from when a story is started to when it is accepted by the product owner or stakeholders. It often also means the time from when a story first exists to the time it is delivered to the end users.

Delivery team, development team The delivery or development team delivers the software that meets business goals and provides value

to customers. Everyone involved in delivering software is a developer, including programmers, testers, business analysts, database experts, system administrators, technical writers, architects, user experience design experts, and operations staff.

DevOps DevOps, a term that combines development and operations, refers to practices and patterns that improve collaboration among different roles in delivery teams, streamlining the process of delivering high-quality software. These practices and patterns help teams write testable, maintainable code, package the product, deploy it, and support the deployed code.

End game The end game is the time before release when the delivery team applies the finishing touches to the product. Not a bug-fixing or “hardening” period, it is an opportunity to work with groups outside of development to help move the software into production. Examples of end game activities include additional testing of database migrations and installation.

Enterprise solutions Software used to satisfy the needs of an organization rather than individual users, mostly found in large organizations. The software is intended to solve an enterprise-wide problem, rather than a departmental problem. Enterprise-level software aims to improve the enterprise’s productivity and efficiency by providing business logic support functionality (Wikipedia, 2014d).

Exploratory testing (ET) Exploratory testing combines test design with test execution and focuses on learning about the application under test. Please see Chapter 12, “Exploratory Testing,” for a detailed guide.

Extract, transform, and load (ETL) Extract, transform, and load refers to a process in database usage, especially for data warehouses, that extracts data from outside sources, transforms it to fit operational needs, and loads it into a target database (Wikipedia, 2014e).

Federated data Data federation technology is software that provides an organization with the ability to aggregate data from disparate sources in a virtual database so it can be used for business intelligence or other analysis (Search Data Management, 2010).

Heuristics Heuristics refers to experience-based techniques for problem solving, learning, and discovery that give a solution that is not guaranteed to be optimal. When an exhaustive search is impractical, heuristic methods are used to speed up the process of finding a satisfactory solution via mental shortcuts to ease the cognitive load of making a decision. Examples of this method include using a rule of thumb, an educated guess, an intuitive judgment, stereotyping, or common sense (Wikipedia, 2014g).

Infrastructure as a service (IaaS) Infrastructure as a service is a provision model in which an organization outsources the equipment used to support operations, including storage, hardware, servers, and networking components. The service provider owns the equipment and is responsible for housing, running, and maintaining it. The client typically pays on a per-use basis (SearchCloudComputing, 2010a).

Kanban The term *kanban* is Japanese and has the literal meaning of “card” or “signboard.” In the Toyota Production System, it designates a system used for “just in time” inventory control (Agile Alliance, 2012). In agile software development, the “kanban method” is characterized by limits on work in progress, the idea of “pulling” work instead of having it assigned by managers, and a self-directed team that manages its own sustainable workload. Kanban boards are used to visualize projects and help enforce work in progress (WIP) limits (Hiranabe, 2008).

Legacy system A legacy system is one that is not supported by automated regression tests. Introducing changes in legacy code, or refactoring it, might be risky because there are no tests to catch unintended changes in system behavior.

Minimum viable product (MVP) The term *minimum viable product* was coined by Frank Robinson and popularized by Eric Ries for software products (Wikipedia, 2014i). A minimum viable product has just enough core features to allow the product to be deployed to a small subset of potential customers. These users form part of the “build-measure-learn” or “validated learning” feedback loop, guiding further development and decision making. An MVP is also used to represent the least amount of change that can be delivered to the customer that adds business value.

Operational acceptance testing (OAT) OAT is used to conduct operational readiness (prerelease) of a product, service, or system as part of a quality management system. OAT is a common type of nonfunctional software testing, used mainly in software support and software maintenance projects.

Platform as a service (PaaS) An outgrowth of software as a service, PaaS is a service delivery model that allows the customer to rent virtualized servers and associated services for running existing applications or developing and testing new ones (SearchCloudComputing, 2010).

Product release A software product release may consist of a brand-new product or new features for an existing product. In agile development, a feature may consist of one or more completed user stories.

Refactoring Refactoring is changing code, without changing its functionality, to make it more maintainable, easier to read, easier to test, and easier to extend.

Regression test A regression test verifies that the behavior of the system under test hasn't changed. Regression tests are usually written as unit tests to drive coding, or acceptance tests to define desired system behavior. Once the tests pass, they become part of a regression test suite, to guard against unintended changes being introduced. Regression tests should be automated to ensure continual feedback.

Release candidate A release candidate is a version or build of a product that can potentially be released to production. The release candidate may undergo further testing or be augmented with documentation or other materials.

Return on investment (ROI) Return on investment, a term borrowed from the world of financial investments, is a measure of the efficiency of an investment. ROI can be calculated in different ways, but it's basically the difference between the gain from an investment and the cost of that investment, divided by the cost of that investment. In testing, ROI is the benefit gained from a testing activity, such as automating a test, weighed against the cost of producing and maintaining that test.

Set-based development As applied to software development, in set-based development, two development pairs or teams independently devise a problem solution or feature. Stakeholders, whether the business or other development team members, evaluate the results and choose the one they prefer.

Software as a service (SaaS) In the software-as-a-service model, applications are hosted by a vendor or service provider. Instead of installing the software on their own hardware, as they would with “software as a product,” the clients access the software on the provider’s servers via a network, usually the Internet (SearchCloudComputing, 2010c).

Source code control system Also referred to as version control, revision control, or source code management, source code control systems allow multiple team members to work on the same code base, documents, or other types of files without stepping on one another’s updates. Common features include merging changes to the same file, reverting changes, comparing versions, viewing a history of changes, and retrieving specific versions. Today, hosting services for distributed code repositories such as GitHub, which uses Git as its revision control system, are popular (Wikipedia, 2014j).

Specification by example (SBE) Specification by example uses process patterns that help software delivery teams collaborate with customer teams to define the scope of work to achieve business goals. SBE illustrates specifications with concrete examples, refines the specifications from key examples, and then automates them to validate the specifications frequently during development. The validated executable specifications are kept as living documentation (Adzic, 2011).

Technical debt Ward Cunningham first introduced this metaphor, applying the concept of financial debt to software. When software teams cut some corners and sacrifice quality in order to help the business take advantage of an opportunity, they incur debt in the form of software that’s not well designed, not protected by automated regression tests, or lacking in other quality characteristics. If the team doesn’t take time to “repay” the debt by refactoring the code or adding automated regression tests, it becomes harder to deliver new features, and the “compounded” debt slows the team down.

Test-driven development (TDD) In test-driven development, the programmer writes and automates a small unit test, which initially fails, before writing the minimum amount of code that will make the test pass. The code is refactored as needed to meet acceptable standards. The production code is made to work one test at a time. TDD, also known as test-driven design, is more of a code design practice than a testing activity and helps build robust, easily maintainable code.

Unit test A unit test verifies the behavior of a tiny part of the overall system, such as a single branch of a single function. It may be as small as a single object or method that is a consequence of one or more design decisions. In an agile context, automated unit tests guide coding at a low level. A unit test is typically less than a dozen lines of code. Some practitioners prefer other terms such as *microtest* (Hill, 2009) or *xunit testing* (Fowler, 2014).

This page intentionally left blank

REFERENCES

Adzic, Gojko, *Bridging the Communication Gap: Specification by Example and Agile Acceptance Testing*, Neuri Limited, 2009.

———, “How Google Does Test Engineering,” <http://gojko.net/2010/10/15/how-google-does-test-engineering/>, 2010a.

———, “How to Implement UI Testing without Shooting Yourself in the Foot,” <http://gojko.net/2010/04/13/how-to-implement-ui-testing-without-shooting-yourself-in-the-foot-2/>, 2010b.

———, *Specification by Example: How Successful Teams Deliver the Right Software*, Manning, 2011.

———, *Impact Mapping: Making a Big Impact with Software Products and Projects*, Provoking Thoughts Ltd., 2012.

———, “The February Revolution,” <http://gojko.net/2013/02/13/the-february-revolution>, 2013.

Adzic, Gojko, Declan Whelan, et al., “Specification by Example” diagram, <https://docs.google.com/drawings/d/1cbfKq-KazcbMVCnRfih6zMSDBdtf90KviV7l2oxGyWM/edit?hl=en>, Agile Alliance aa-fft workshop, 2010.

Agile Alliance, “Kanban Board,” <http://guide.agilealliance.org/guide/kanban.html>, 2012.

Andrea, Jennitta, “Acceptance Test-Driven Development: Not as Optional as You Think,” www.stickyminds.com/article/acceptance-test-driven-development-not-optional-you-think, StickyMinds, Iterations newsletter, 2010.

Appelo, Jurgen, *Management 3.0: Leading Agile Developers, Developing Agile Leaders*, Addison-Wesley, 2011.

———, “Feedback Wrap,” www.management30.com/workout/feedback-wrap/, 2013.

Ariely, Dan, “Are We in Control of Our Own Decisions?” www.ted.com/talks/dan_ariely_asks_are_we_in_control_of_our_own_decisions, 2008.

Bach, Jon, “Session-Based Test Management,” www.satisfice.com/articles/sbtm.pdf, *Software Testing and Quality Engineering* magazine, now *Better Software* magazine, 2000.

Beck, Kent, *Test-Driven Development: By Example*, Addison-Wesley, 2002.

Bell, Rob, “A Beginner’s Guide to Big O Notation,” <http://rob-bell.net/2009/06/a-beginners-guide-to-big-o-notation/>, 2009.

Brodwell, Johannes, “Remote Pair Programming,” www.slideshare.net/jhannes/2013-0807-agile-2013-remote-pair-programming, August 2013.

CMMI Institute, Carnegie Mellon, “Solutions,” <http://cmmiinstitute.com/cmmi-solutions/>, 2014.

Cockburn, Alistair, “Characterizing People as Non-Linear, First-Order Components in Software Development,” <http://alistair.cockburn.us/Characterizing+people+as+non-linear%2c+first-order+components+in+software+development>, 1999.

———, “Hexagonal Architecture: The Pattern: Ports and Adapters,” <http://alistair.cockburn.us/Hexagonal+architecture>, 2005.

Crispin, Lisa, and Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*, Addison-Wesley, 2009.

Cunningham, Ward, “Debt Metaphor,” www.youtube.com/watch?v=pqeJFYwnkJE, 2009.

De Bono, Edward, *Thinking for Action*, DK Publishing, 1998.

Derby, Esther, Johanna Rothman, and Gerald M. Weinberg, “Problem-Solving Leadership Course,” www.estherderby.com/problem-solving-leadership-psl, 2014.

Dinwiddie, George, “A Lingua Franca between the Three (or More) Amigos,” <http://blog.gdinwiddie.com/2010/02/25/a-lingua-franca-between-the-three-or-more-amigos/>, 2010.

Eliot, Seth, “A to Z Testing in Production: TiP Methodologies, Techniques and Examples at STPCon 2012,” www.setheliot.com/blog/a-to-z-testing-in-production-tip-methodologies-techniques-and-examples-at-stpcon-2012, 2012.

Emery, Dale, “Writing Maintainable Automated Acceptance Tests,” dhemery.com/pdf/writing_maintainable_automated_acceptance_tests.pdf, 2009.

Evans, David, “Visualising Quality,” <http://prezi.com/yych7sndetph/visualising-quality-atd>, 2013.

Fowler, Martin, “UnitTest,” <http://martinfowler.com/bliki/UnitTest.html>, 2014.

Freeman, Steve, and Nat Pryce, *Growing Object-Oriented Software, Guided by Tests*, Addison-Wesley, 2009.

Gärtner, Markus, “Technical Debt Applied to Testing,” www.shino.de/2009/01/08/technical-debt-applied-to-testing/, 2009.

———, “Pomodoro Testing,” www.shino.de/tag/pomodoro-testing, 2011.

———, *ATDD by Example: A Practical Guide to Acceptance Test-Driven Development*, Addison-Wesley, 2012.

———, personal communication, 2014.

Gawande, Atul, “Slow Ideas,” *The New Yorker*, July 29, 2013.

Gilb, Tom, and Kai Gilb, “Planguage,” www.gilb.com/definitionPlanguage&structure=Glossary&page_ref_id=476, 2013.

Gottesdiener, Ellen, *Requirements by Collaboration: Workshops for Defining Needs*, Addison-Wesley, 2002.

———, *The Software Requirements Memory Jogger: A Pocket Guide to Help Software and Business Teams Develop and Manage Requirements*, Goal QPC Inc., 2005.

Gottesdiener, Ellen, and Mary Gorman, *Discover to Deliver: Agile Product Planning and Analysis*, 2012.

Gregory, Janet, “About Learning 2,” <http://janetgregory.ca/about-learning-2/>, 2010.

———, “Jigsaw Puzzles and Small Chunks,” <http://janetgregory.ca/jigsaw-puzzles-small-chunks/>, 2011.

———, “Distributed Teams—My Experience,” <http://janetgregory.ca/distributed-teams-my-experience/>, 2014.

Guest, David, “The Hunt Is On for the Renaissance Man of Computing,” *The Independent* (London), 1991.

Hagar, Jon, *Software Test Attacks to Break Mobile and Embedded Devices*, Chapman and Hall/CRC, 2013.

———, personal communication, 2014.

Hagberg, Jan Petter, personal communication, 2013.

Haines, Corey, et al., Coderetreat Community Network, <http://coderetreat.org/>.

Harty, Julian, “Trinity Testing,” <http://julianharty-softwaretesting.blogspot.ca/2010/05/trinity-testing.html>, 2010.

———, *Testing and Test Automation for Mobile Phone Applications*, Chapman and Hall/CRC, 2015.

Hassa, Christian, “Story Maps in Practice,” www.slideshare.net/chassa/2013-0509story-mapsagilemeetupbp, 2013.

Hendrickson, Elisabeth, “Testing: Not a Phase, but a Way of Life,” <http://testobsessed.com/2006/11/testing-not-a-phase-but-a-way-of-life/>, 2006.

———, “Driving Development with Tests: ATDD and TDD,” <http://testobsessed.com/wp-content/uploads/2011/04/atddexample.pdf>, 2008.

———, “The Agile Acid Test,” <http://testobsessed.com/2010/12/the-agile-acid-test/>, 2010.

———, “Test Heuristics Cheat Sheet,” <http://testobsessed.com/wp-content/uploads/2011/04/testheuristicscheatsheetv1.pdf>, 2011.

———, “The Thinking Tester: Evolved,” www.slideshare.net/ehendrickson/the-thinking-tester-evolved, 2012.

———, *Explore It! Reduce Risk and Increase Confidence with Exploratory Testing*, Pragmatic Programmer, 2013.

Heusser, Matthew, *How to Reduce the Cost of Software Testing*, Taylor & Francis, 2011.

Hill, Michael, “They’re Called Microtests,” <http://anarchycreek.com/2009/05/20/theyre-called-microtests/>, 2009.

Hiranabe, Kenji, “Kanban Applied to Software Development: From Agile to Lean,” www.infoq.com/articles/hiranabe-lean-agile-kanban, InfoQ, 2008.

Humble, Jez, “There’s No Such Thing as a ‘DevOps Team,’” <http://continuousdelivery.com/2012/10/theres-no-such-thing-as-a-devops-team/>, 2012.

Humble, Jez, and David Farley, *Continuous Delivery: Reliable Software Releases through Build, Test and Deployment Automation*, Addison-Wesley, 2010.

Hunt, Andy, *Pragmatic Thinking and Learning*, Pragmatic Bookshelf, 2008.

Hussman, David, “Agile Journeys,” www.slideshare.net/agileee/agile-journeys-by-david-hussman-2029404, 2009.

———, “Cutting an Agile Groove: A Free Educational Video Series for Newbies and Practitioners,” <http://pragprog.com/screencasts/v-dhcag/cutting-an-agile-groove>, Pragmatic Bookshelf, 2011.

———, “Story Maps, Slices, Customer Journeys, and Other Product Design Tools,” <http://agilepalooza.com/austin2013/slidedecks/Story-Maps-Slices-Customer-Journeys.pdf>, 2013.

Hüttermann, Michael, *Agile ALM: Lightweight Tools and Agile Strategies*, Manning Publications, 2011a.

———, “Agile ALM and Collaborative Development,” <http://huettermann.net/perform/AgileALM-AgileRecord-Huettermann.pdf>, Agile Record, 2011b.

———, *DevOps for Developers: Integrate Development and Operations, the Agile Way*, Apress, 2012.

Kahneman, Daniel, *Thinking, Fast and Slow*, Farrar, Straus and Giroux, 2011.

Kaner, Cem, “What Is Context-Driven Testing?” <http://kaner.com/?p=49>, 2012.

Karten, Naomi, “Are You Listening?” www.agileconnection.com/article/are-you-listening, Agile Connection, 2009.

Kemerling, Ashton, “Generative Testing,” www.pivotaltracker.com/community/tracker-blog/generative-testing, Pivotal, 2014.

Keogh, Liz, Yahoo Agile Testing group post, August 2010.

———, “Behaviour-Driven Development,” www.slideshare.net/lunivore/behavior-driven-development-11754474, 2012a.

———, “The Deliberate Discovery Workshop,” <http://lizkeogh.com/2012/09/21/the-deliberate-discovery-workshop/>, 2012b.

———, “BDD before the Tools,” <http://lizkeogh.com/2013/10/24/bdd-before-the-tools/>, 2013a.

———, “Embracing Uncertainty,” including links to posts on Cynefin, Deliberate Discovery, and Real Options, <http://lizkeogh.com/embracing-uncertainty/>, 2013b.

———, “Discrete vs. Continuous Capabilities,” <http://lizkeogh.com/2014/02/10/discrete-vs-continuous-capabilities/>, 2014a.

Keogh, Liz, personal communication, 2014b.

Kniberg, Henrik, and Spotify Labs, “Spotify Culture,” <http://labs.spotify.com/2014/03/27/spotify-engineering-culture-part-1/>, 2013.

Kniberg, Henrik, and Anders Ivarsson, “Scaling Agile @ Spotify: With Tribes, Squads, Chapters & Guilds,” <http://ucvox.files.wordpress.com/2012/11/113617905-scaling-agile-spotify-11.pdf>, 2012.

Knight, Adam P., “The Thread of an Idea: Adopting a Thread-Based Approach to Exploratory Testing,” www.a-sisyphean-task.com/2011/11/thread-based-approach-to-exploratory.html, 2011.

———, “Fractal Exploratory Testing,” www.a-sisyphean-task.com/2013/01/fractal-exploratory-testing.html, 2013.

———, personal communication, 2014.

Kohl, Jonathan, *Tap into Mobile Testing*, LeanPub, 2013.

Kubasek, Stanley, “The Boy Scout Rule,” <http://pragmaticcraftsman.com/2011/03/the-boy-scout-rule/>, The Pragmatic Craftsman, 2011.

Lambert, Rob, “T-Shaped Testers and Their Role in a Team,” <http://thesocialtester.co.uk/t-shaped-testers-and-their-role-in-a-team/>, 2012.

Levison, Mark, “The Beginner’s Mind—An Approach to Listening,” www.infoq.com/news/2008/08/beginners_mind, InfoQ, 2008.

Lyndsay, James, “Adventures in Session-Based Testing,” www.workroom-productions.com/papers/AiSBTV1.2.pdf, 2003.

———, “Why Exploration Has a Place in Any Strategy,” www.workroom-productions.com/papers/Exploration%20and%20Strategy.pdf, 2006.

———, “Testing in an Agile Environment,” www.workroom-productions.com/papers/Testing%20in%20an%20agile%20environment.pdf, 2007.

———, personal communication, 2014.

Manns, Mary Lynn, and Linda Rising, *Fearless Change: Patterns for Introducing New Ideas*, Addison-Wesley, 2005.

Marick, Brian, “Agile Testing Directions: Test and Examples,” www.exampler.com/old-blog/2003/08/22/#agile-testing-project-2, 2003.

———, *Everyday Scripting with Ruby: For Teams, Testers and You*, Pragmatic Bookshelf, 2007.

Martin, Robert C., *Clean Code: A Handbook of Software Craftsmanship*, Prentice Hall, 2009.

———, *The Clean Coder: A Code of Conduct for Professional Programmers*, Prentice Hall, 2011.

Matts, Chris, and Gojko Adzic, “Feature Injection: Three Steps to Success,” www.infoq.com/articles/feature-injection-success, InfoQ, 2011.

Matts, Chris, and Olav Maassen, “‘Real Options’ Underlie Agile Practices,” www.infoq.com/articles/real-options-enhance-agility, InfoQ, 2007.

McDonald, Kent J., “Create a Vendor Contract While Keeping Agile,” www.techwell.com/2013/05/create-vendor-contract-while-keeping-agile, Techwell, 2013.

McDonald, Mark, “Is Your Company an Enterprise? The Answer Matters,” http://blogs.gartner.com/mark_mcdonald/2009/06/15/is-your-company-an-enterprise-the-answer-matters/, Gartner Blog Network, 2009.

McMahon, Chris, “Telecommuting Policy,” <http://chrismcmahonsblog.blogspot.com/2009/12/telecommuting-policy.html>, 2009.

Morgan, Jeff “Cheezy,” “UI Tests—Default Data,” www.cheezyworld.com/2010/11/21/ui-tests-default-dat, 2010.

———, *Cucumber and Cheese: A Testers Workshop*, LeanPub, 2013.

———, personal communication, 2014.

Morville, Peter, “User Experience Design,” <http://semanticstudios.com/publications/semantics/000029.php>, 2004.

Nordstrom, “Nordstrom Innovation Lab: Sunglass iPad App Case Study,” www.youtube.com/watch?v=szr0ezLyQHY, 2011.

North, Dan, “Introducing BDD,” <http://dannorth.net/introducing-bdd/>, *Better Software* magazine, 2006.

Ottinger, Tim, and Jeff Langr, “Arrange-Act-Assert,” <http://agileinaflash.blogspot.com/2009/03/arrange-act-assert.html>, 2009a.

———, *Agile in a Flash: Speed Learning Agile Software*, Pragmatic Bookshelf, 2011.

———, “FURPS” model, <http://agileinaflash.blogspot.ca/2009/04/furps.html>, 2009b.

———, “Red-Green-Refactor,” <http://agileinaflash.blogspot.co.uk/2009/02/red-green-refactor.html>, 2009c.

Patton, Jeff, “It’s All in How You Slice It: Design Your Project in Working Layers to Avoid Half-Baked Incremental Releases,” www.agileproductdesign.com/writing/how_you_slice_it.pdf, *Better Software* magazine, 2005.

———, “Pragmatic Personas,” www.stickyminds.com/article/pragmatic-personas, StickyMinds/Techwell, 2010.

———, *User Story Mapping: Building Better Products Using Agile Software Design*, O’Reilly Media, 2014.

Rainsberger, J. B., “The Next Decade of Agile Software Development,” www.slideshare.net/jbrains/the-next-decade-of-agile-software-development, 2013.

Ramdeo, Anand, “Test Automation—How to Handle Common Components with Page Object Model?,” www.testinggeek.com/test-automation-how-to-handle-common-components-with-page-object-model, 2013.

Rasmussen, Jonathan, *The Agile Samurai: How Agile Masters Deliver Great Software*, Pragmatic Bookshelf, 2010.

Ries, Eric, “Case Study: SlideShare Goes Freemium,” www.startuplessonslearned.com/2010/08/case-study-slideshare-goes-freemium.html, 2010.

Rising, Linda, “The Power of Retrospectives,” www.stickyminds.com/presentation/power-retrospectives-0, SQE, 2009.

Rogalsky, Steve, “Thoughts on Beyond Deadlines by Jabe Bloom,” <http://winnipegagilist.blogspot.ca/2013/03/thoughts-on-beyond-deadlines-by-jabe.html>, 2012.

Rothman, Johanna, “Agile Is Not for Everyone,” www.jrothman.com/blog/mpd/2012/12/agile-is-not-for-everyone.html, 2012a.

———, *Hiring Geeks That Fit*, Pragmatic Bookshelf, 2012b.

Ruhland, Bernice Niel, “Developing Your Leadership Skills through a Journal Club,” <http://thetestersedge.com/2013/12/28/developing-your-leadership-skills-through-a-journal-club/>, 2013a.

———, personal communication, 2013b.

———, personal communication, 2014.

Satir Global Network, <http://satirglobal.org/>.

Scott, Alistair, “Specification by Example: A Love Story,” <http://watirmelon.files.wordpress.com/2011/05/specificationbyexamplealovestory1.pdf>, 2011a.

———, “Yet Another Software Testing Pyramid,” <http://watirmelon.com/2011/06/10/yet-another-software-testing-pyramid/>, 2011b.

SearchCloudComputing, “Infrastructure as a Service,” <http://searchcloudcomputing.techtarget.com/definition/Infrastructure-as-a-Service-IaaS>, 2010a.

———, “Platform as a Service,” <http://searchcloudcomputing.techtarget.com/definition/Platform-as-a-Service-PaaS>, 2010b.

———, “Software as a Service,” <http://searchcloudcomputing.techtarget.com/definition/Software-as-a-Service>, 2010c.

Search Data Management, “Data Federation Technology,” <http://searchdatamanagement.techtarget.com/definition/data-federation-technology>, 2010.

Sherry, Rosie, Ministry of Testing’s mobile testing mind map, www.flickr.com/photos/softwaretestingclub/7159412943/sizes/o/in/photostream/, Ministry of Testing, 2013.

Sweets, Tony, “Page Object Example,” <https://github.com/tsweets/page-objects>, 2013.

Talks, Mike, *The Software Minefield*, LeanPub, 2012.

Traynor, Des, “An Interview with Andy Budd,” <http://insideintercom.io/an-interview-with-andy-budd/>, Intercom, 2011.

Tung, Portia, “Power of Play,” www.slideshare.net/portiatung/the-powerofplay36, 2011.

Waters, John K., “An Agile Approach to Rocket Science,” <http://adtmag.com/articles/2004/10/06/an-agile-approach-to-rocket-science.aspx>, *Application Development Trends* magazine, October 16, 2004.

Whittaker, James, “ACC Explained,” <http://code.google.com/p/test-analytics/wiki/AccExplained>, 2011.

———, “Exploratory Testing Tours,” http://msdn.microsoft.com/en-us/library/jj620911.aspx#bkmk_tours, Microsoft, 2012.

Whittaker, James A., Jason Arbon, and Jeff Carollo, *How Google Tests Software*, Addison-Wesley, 2012.

Wikipedia, “5 Whys,” http://en.wikipedia.org/wiki/5_Whys, 2014a.

———, “Contextual Inquiry,” http://en.wikipedia.org/wiki/Contextual_inquiry, 2014b.

———, “Embedded System: History,” http://en.wikipedia.org/wiki/Embedded_system#History, 2014c.

———, “Enterprise Software,” http://en.wikipedia.org/wiki/Enterprise_software, 2014d.

———, “Extract, Transform, Load,” https://en.wikipedia.org/wiki/Extract,_transform,_load, 2014e.

———, “FURPS,” <http://en.wikipedia.org/wiki/FURPS>, 2014f.

———, “Heuristics,” <http://en.wikipedia.org/wiki/Heuristics>, 2014g.

———, “Ishikawa Diagram,” http://en.wikipedia.org/wiki/Ishikawa_diagram, 2014h.

———, “Minimum Viable Product,” http://en.wikipedia.org/wiki/Minimum_viable_product, 2014i.

———, “Revision Control,” http://en.wikipedia.org/wiki/Revision_control, 2014j.